



Review Class

Review Class

• Insertion Sort

• Simulation

- Check the slides
- Check the solved problem

• Time Complexity

- Best and Worst Case

• Pseudocode

• Counting Sort

• Simulation

- Check the slides
- Check the solved problem

• Time Complexity

• Pseudocode

Review Class

- **Array Mapping**

- Understand the formulae
- Practice



- **Binary Search**

- Time Complexity

- Simulation
 - See the slides
 - See the solved problem
- Pseudocode

Review Class

- **Merge Sort**

- Simulation using tree
 - Slide
 - Solved problem
- Time complexity
- Pseudocode

- **Quick Sort**

- Simulation of Partition
- Time Complexity
- Pseudocode

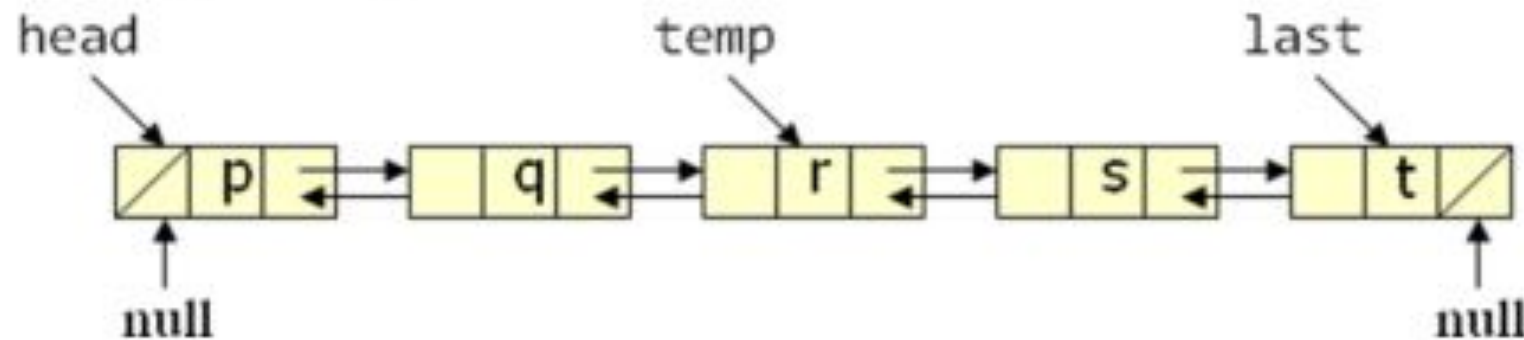
Review Class - Linked List

- Practice the functions taught in class
 - Insertion
 - Deletion
 - Search
- Writing Code and Simulation
 - Write the code for a certain scenario (Check solved problems)
 - Show the results of a code (Check solved problems)
 - Advantages and Disadvantages

Review Class - Linked List

a) Answer the following questions for the doubly linked list as shown below, where $p = \text{last two digits of your student id} + 9$, $q = p+4$, $r = p+q$, $s = r-3$, $t = r+s$.

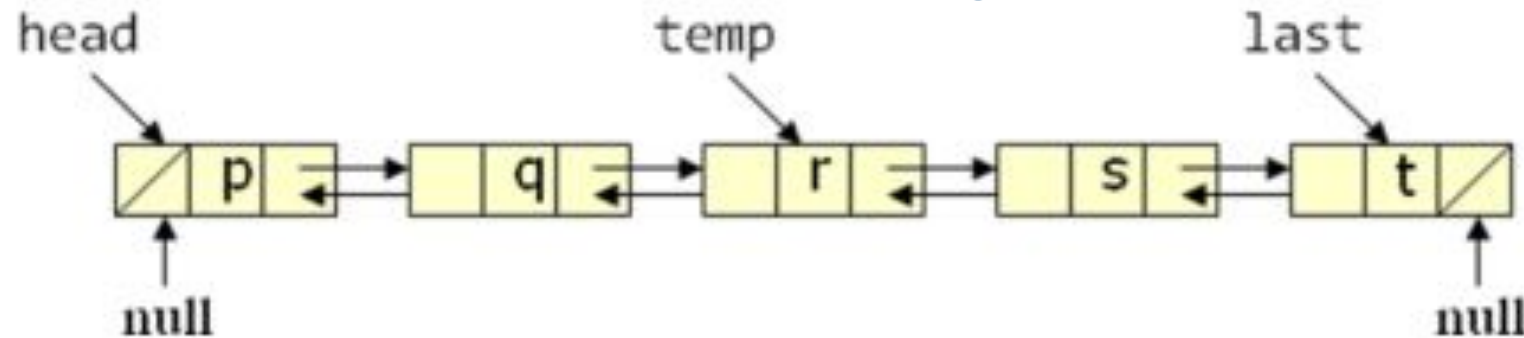
- a) $\text{head} \rightarrow \text{next} \rightarrow \text{next} \rightarrow \text{value} = ?$
- b) $\text{last} \rightarrow \text{prev} \rightarrow \text{next} \rightarrow \text{value} = ?$
- c) $\text{temp} \rightarrow \text{prev} \rightarrow \text{prev} \rightarrow \text{prev} = ?$
- d) $\text{temp} \rightarrow \text{next} \rightarrow \text{prev} \rightarrow \text{prev} \rightarrow \text{value} = ?$
- e) $\text{last} \rightarrow \text{prev} \rightarrow \text{prev} \rightarrow \text{next} \rightarrow \text{value} = ?$



Review Class - Linked List

a) Answer the following questions for the doubly linked list as shown below, where $p =$ last two digits of your student id + 9, $q = p+4$, $r = p+q$, $s = r-3$, $t = r+s$.

- a) $\text{head} \rightarrow \text{next} \rightarrow \text{next} \rightarrow \text{value} = ?$ r
- b) $\text{last} \rightarrow \text{prev} \rightarrow \text{next} \rightarrow \text{value} = ?$ t
- c) $\text{temp} \rightarrow \text{prev} \rightarrow \text{prev} \rightarrow \text{prev} = ?$ null
- d) $\text{temp} \rightarrow \text{next} \rightarrow \text{prev} \rightarrow \text{prev} \rightarrow \text{value} = ?$ c
- e) $\text{last} \rightarrow \text{prev} \rightarrow \text{prev} \rightarrow \text{next} \rightarrow \text{value} = ?$ s

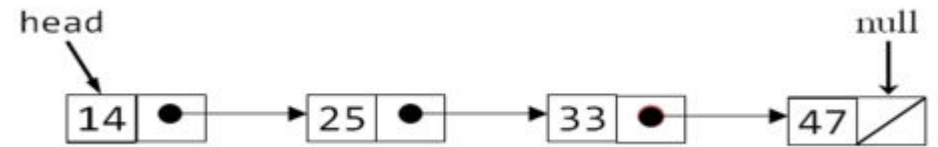


Review Class - Linked List

- b) Assume that you are given a single linked list as shown below.

Write the statements to perform the following:

- i) To insert 40 in between 33 and 47.
- ii) To delete 14 from the list.
- iii) To make a linear circular linked list from the current list.

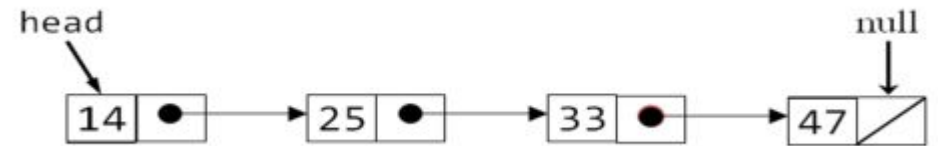


Review Class - Linked List

- b) Assume that you are given a single linked list as shown below.

Write the statements to perform the following:

- i) To insert 40 in between 33 and 47.
- ii) To delete 14 from the list.
- iii) To make a linear circular linked list from the current list.



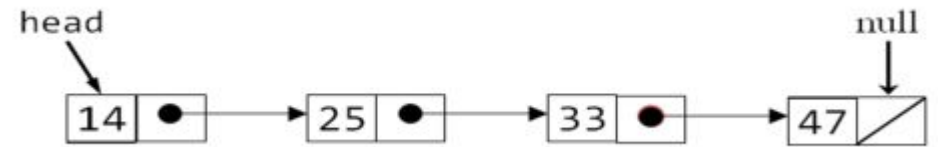
```
struct Node* newNode= malloc(sizeof(struct Node));
newNode->value=40;
newNode->next=head->next->next->next;
head->next->next->next=newNode;
```

Review Class - Linked List

- b) Assume that you are given a single linked list as shown below.

Write the statements to perform the following:

- i) To insert 40 in between 33 and 47.
- **ii) To delete 14 from the list.**
- iii) To make a linear circular linked list from the current list.



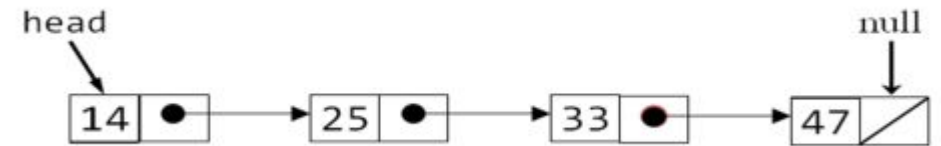
```
temp=head;  
head=head->next;  
free(temp);
```

Review Class - Linked List

- b) Assume that you are given a single linked list as shown below.

Write the statements to perform the following:

- i) To insert 40 in between 33 and 47.
- ii) To delete 14 from the list.
- **iii) To make a linear circular linked list from the current list.**



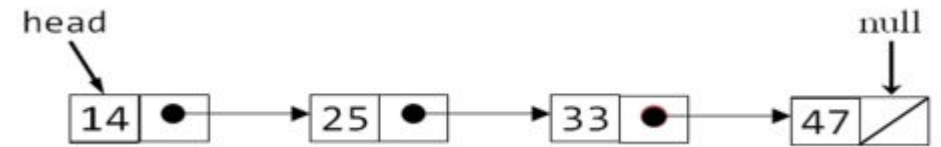
```
head->next->next->next->next=head;
```

Review Class - Linked List

- b) Assume that you are given a single linked list as shown below.

Write the statements to perform the following:

- i) To insert 40 in between 33 and 47.
- ii) To delete 14 from the list.
- **iii) To make a linear circular linked list from the current list.**



```
temp=head;
if(temp==null) {
    return;
}
while(temp->next!=NULL) {
    temp=temp->next;
}

temp->next=head;
```

Review Class Linked List

- `start=(node*)malloc(sizeof(node));`
- `temp=(node*)malloc(sizeof(node));`
- `temp1=(node*)malloc(sizeof(node));`
- `start->data =10;`
- `temp->data=40;`
- `temp1->data=30;`
- `start->next=temp1;`
- `start->next->next=temp;`
- `temp->next=NULL;`
- `start->next=temp1->next;`
- `free(temp1);`
- `newitem=(node*)malloc(sizeof(node));`
- `newitem->data=34;`
- `newitem->next=start->next;`
- `start->next=newitem;`

Show the effect of each of the statements given in the following code segment.

Assume, each of the nodes in the linear linked list has two fields' data and next, where data is of integer type and next will contain the address of the next node.

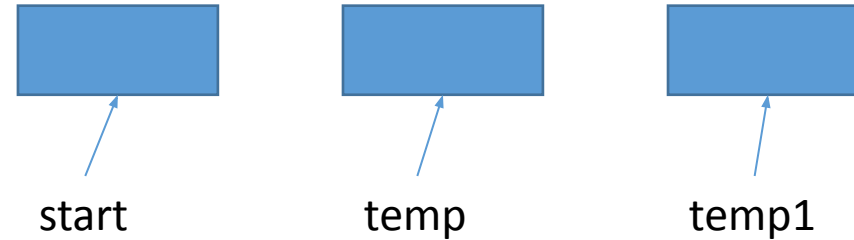
Review Class Linked List

- **start=(node*)malloc(sizeof(node));**
- **temp=(node*)malloc(sizeof(node));**
- **temp1=(node*)malloc(sizeof(node));**
- **start->data =10;**
- **temp->data=40;**
- **temp1->data=30;**
- **start->next=temp1;**
- **start->next->next=temp;**
- **temp->next=NULL;**
- **start->next=temp1->next;**
- **free(temp1);**
- **newitem=(node*)malloc(sizeof(node));**
- **newitem->data=34;**
- **newitem->next=start->next;**
- **start->next=newitem;**



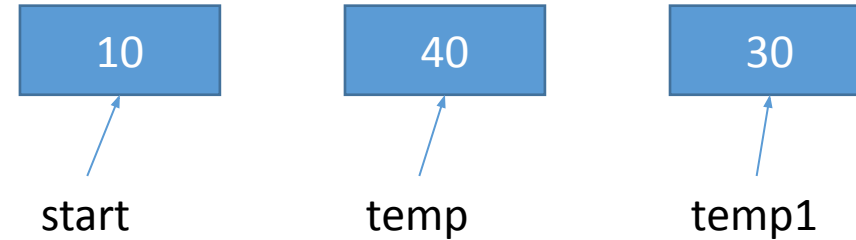
Review Class Linked List

- **start=(node*)malloc(sizeof(node));**
- **temp=(node*)malloc(sizeof(node));**
- **temp1=(node*)malloc(sizeof(node));**
- start->data =10;
- temp->data=40;
- temp1->data=30;
- start->next=temp1;
- start->next->next=temp;
- temp->next=NULL;
- start->next=temp1->next;
- free(temp1);
- newitem=(node*)malloc(sizeof(node));
- newitem->data=34;
- newitem->next=start->next;
- start->next=newitem;



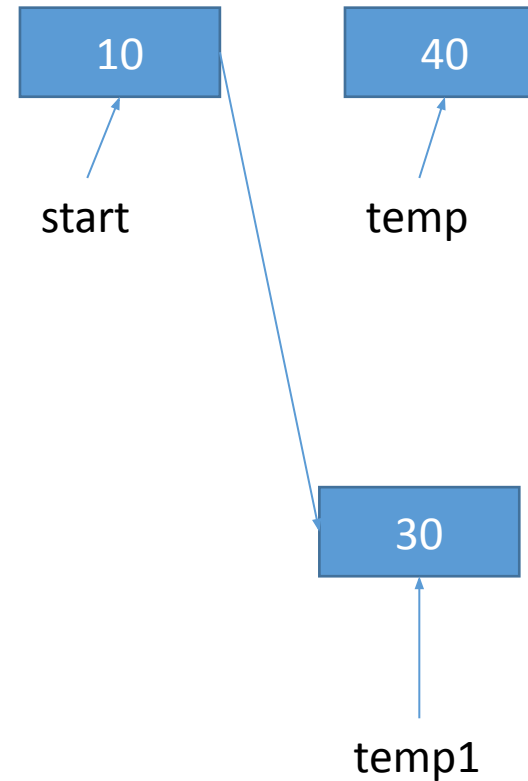
Review Class Linked List

- `start=(node*)malloc(sizeof(node));`
- `temp=(node*)malloc(sizeof(node));`
- `temp1=(node*)malloc(sizeof(node));`
- `start->data =10;`
- `temp->data=40;`
- `temp1->data=30;`
- `start->next=temp1;`
- `start->next->next=temp;`
- `temp->next=NULL;`
- `start->next=temp1->next;`
- `free(temp1);`
- `newitem=(node*)malloc(sizeof(node));`
- `newitem->data=34;`
- `newitem->next=start->next;`
- `start->next=newitem;`



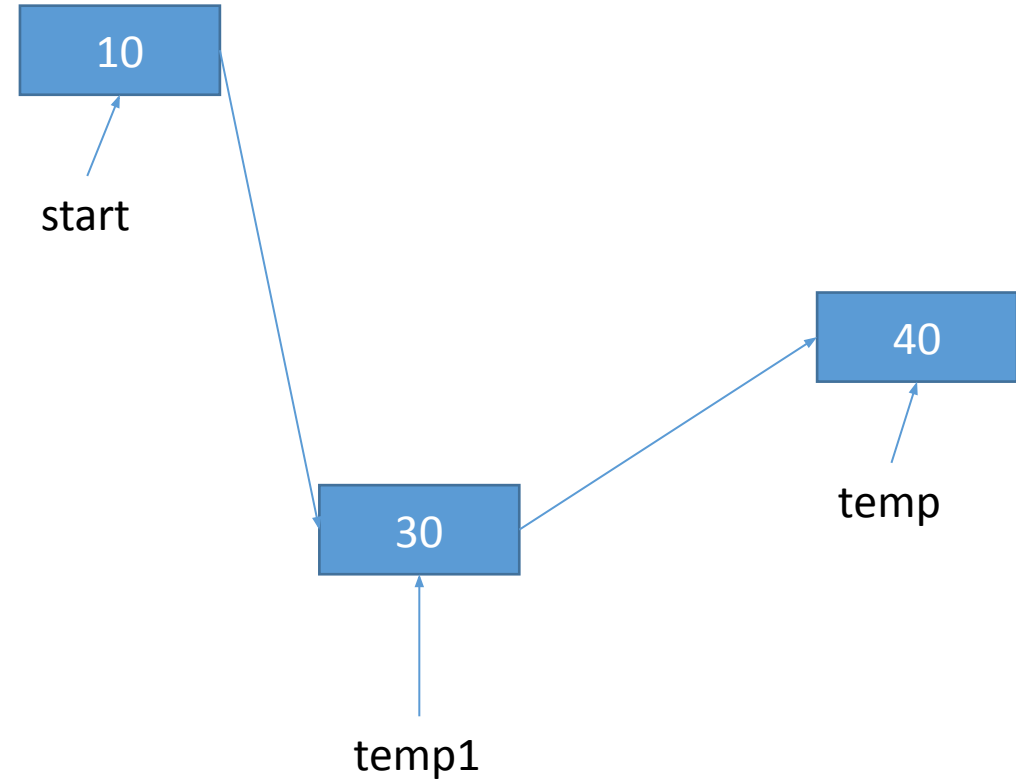
Review Class Linked List

- `start=(node*)malloc(sizeof(node));`
- `temp=(node*)malloc(sizeof(node));`
- `temp1=(node*)malloc(sizeof(node));`
- `start->data =10;`
- `temp->data=40;`
- `temp1->data=30;`
- `start->next=temp1;`
- `start->next->next=temp;`
- `temp->next=NULL;`
- `start->next=temp1->next;`
- `free(temp1);`
- `newitem=(node*)malloc(sizeof(node));`
- `newitem->data=34;`
- `newitem->next=start->next;`
- `start->next=newitem;`



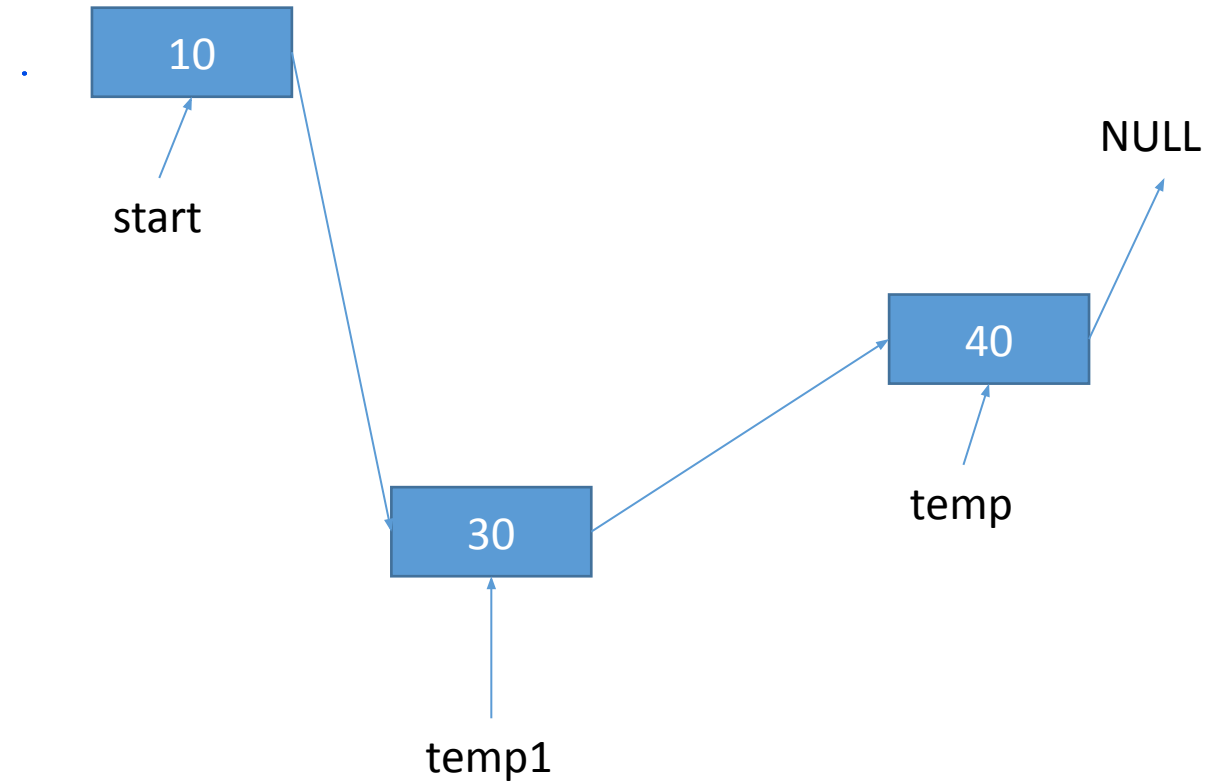
Review Class Linked List

- `start=(node*)malloc(sizeof(node));`
- `temp=(node*)malloc(sizeof(node));`
- `temp1=(node*)malloc(sizeof(node));`
- `start->data =10;`
- `temp->data=40;`
- `temp1->data=30;`
- `start->next=temp1;`
- `start->next->next=temp;`
- `temp->next=NULL;`
- `start->next=temp1->next;`
- `free(temp1);`
- `newitem=(node*)malloc(sizeof(node));`
- `newitem->data=34;`
- `newitem->next=start->next;`
- `start->next=newitem;`



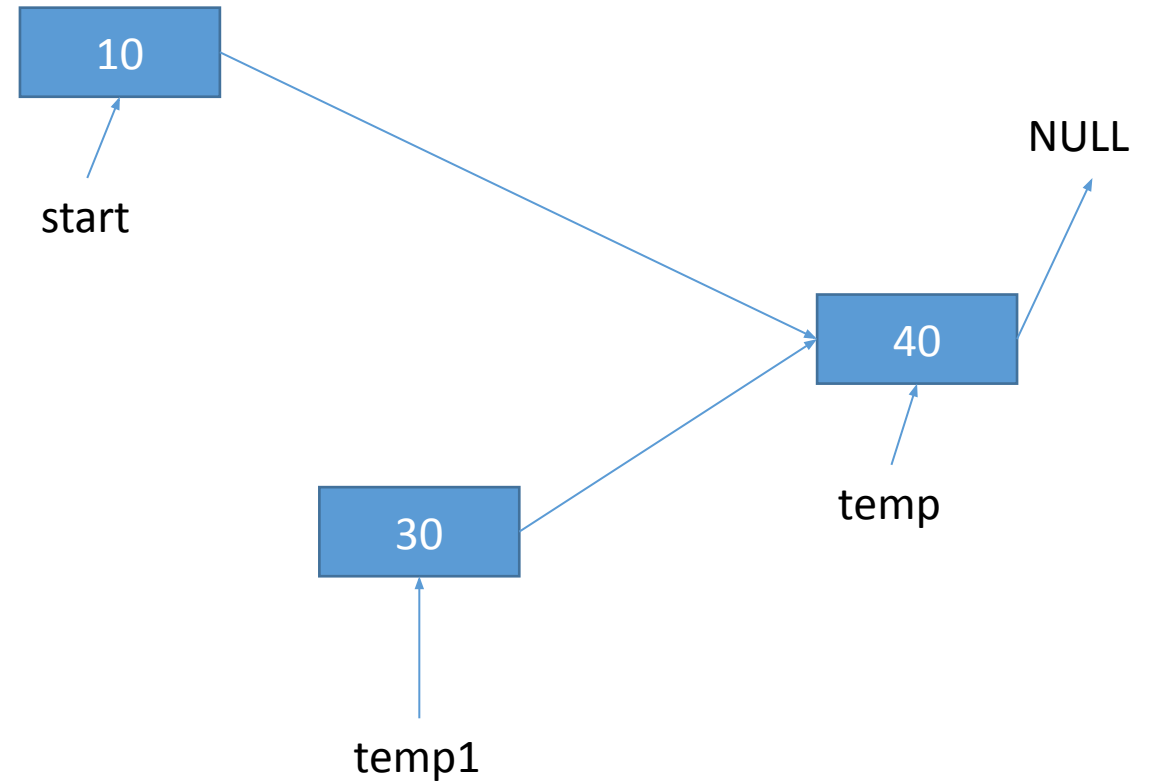
Review Class Linked List

- `start=(node*)malloc(sizeof(node));`
- `temp=(node*)malloc(sizeof(node));`
- `temp1=(node*)malloc(sizeof(node));`
- `start->data =10;`
- `temp->data=40;`
- `temp1->data=30;`
- `start->next=temp1;`
- `start->next->next=temp;`
- `temp->next=NULL;`
- `start->next=temp1->next;`
- `free(temp1);`
- `newitem=(node*)malloc(sizeof(node));`
- `newitem->data=34;`
- `newitem->next=start->next;`
- `start->next=newitem;`



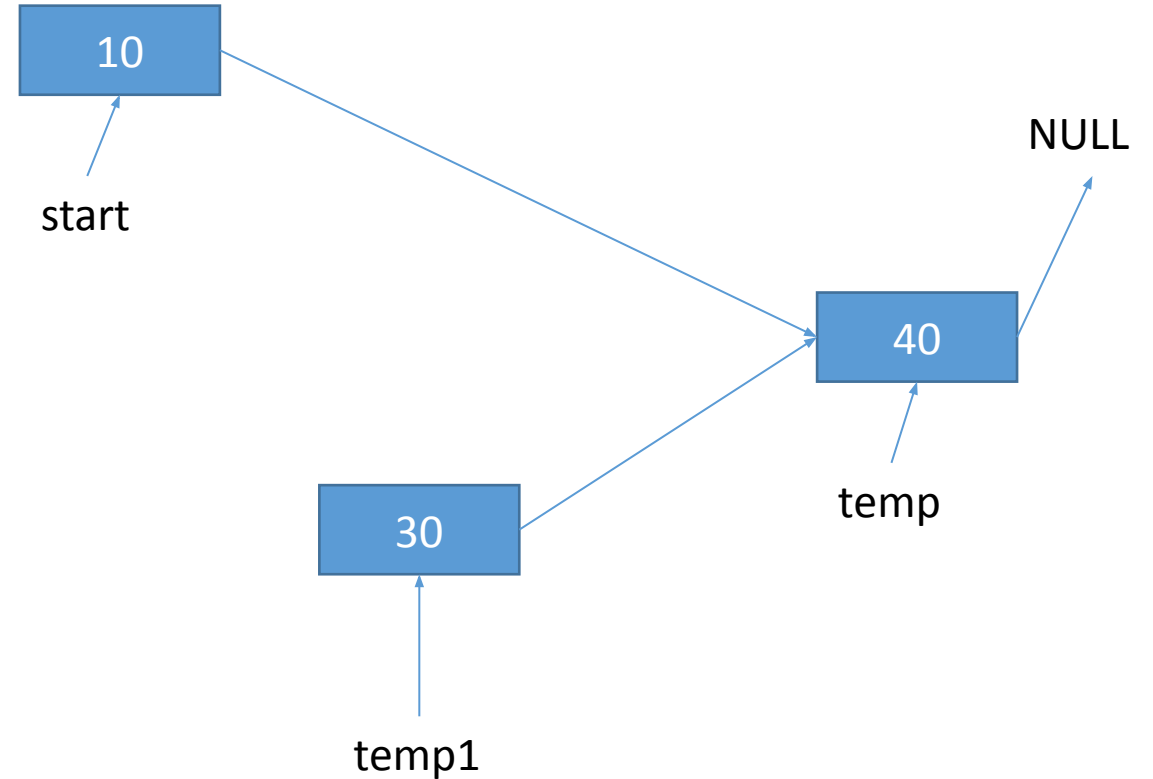
Review Class Linked List

- `start=(node*)malloc(sizeof(node));`
- `temp=(node*)malloc(sizeof(node));`
- `temp1=(node*)malloc(sizeof(node));`
- `start->data =10;`
- `temp->data=40;`
- `temp1->data=30;`
- `start->next=temp1;`
- `start->next->next=temp;`
- `temp->next=NULL;`
- `start->next=temp1->next;`
- `free(temp1);`
- `newitem=(node*)malloc(sizeof(node));`
- `newitem->data=34;`
- `newitem->next=start->next;`
- `start->next=newitem;`



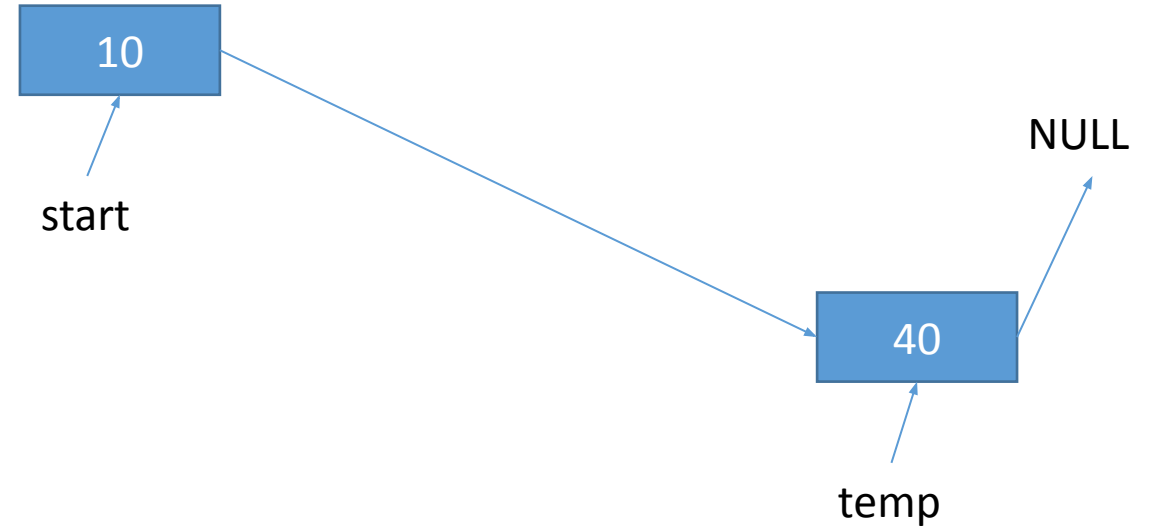
Review Class Linked List

- `start=(node*)malloc(sizeof(node));`
- `temp=(node*)malloc(sizeof(node));`
- `temp1=(node*)malloc(sizeof(node));`
- `start->data =10;`
- `temp->data=40;`
- `temp1->data=30;`
- `start->next=temp1;`
- `start->next->next=temp;`
- `temp->next=NULL;`
- `start->next=temp1->next;`
- `free(temp1);`
- `newitem=(node*)malloc(sizeof(node));`
- `newitem->data=34;`
- `newitem->next=start->next;`
- `start->next=newitem;`



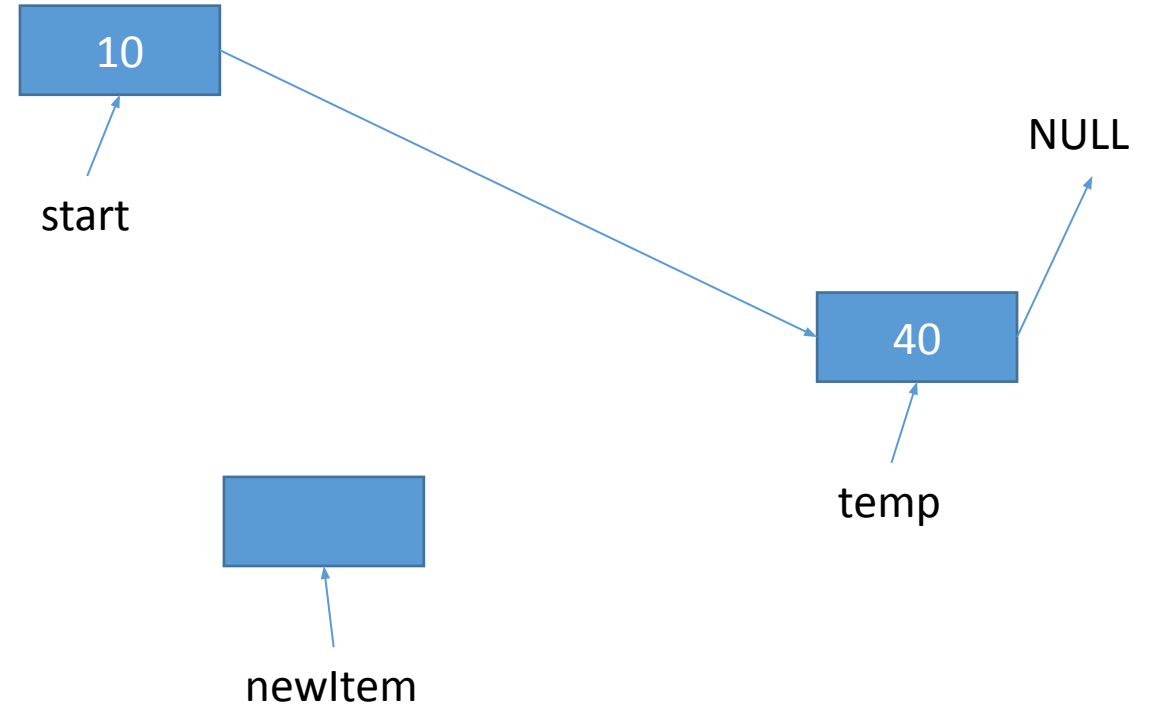
Review Class Linked List

- `start=(node*)malloc(sizeof(node));`
- `temp=(node*)malloc(sizeof(node));`
- `temp1=(node*)malloc(sizeof(node));`
- `start->data =10;`
- `temp->data=40;`
- `temp1->data=30;`
- `start->next=temp1;`
- `start->next->next=temp;`
- `temp->next=NULL;`
- `start->next=temp1->next;`
- `free(temp1);`
- `newitem=(node*)malloc(sizeof(node));`
- `newitem->data=34;`
- `newitem->next=start->next;`
- `start->next=newitem;`



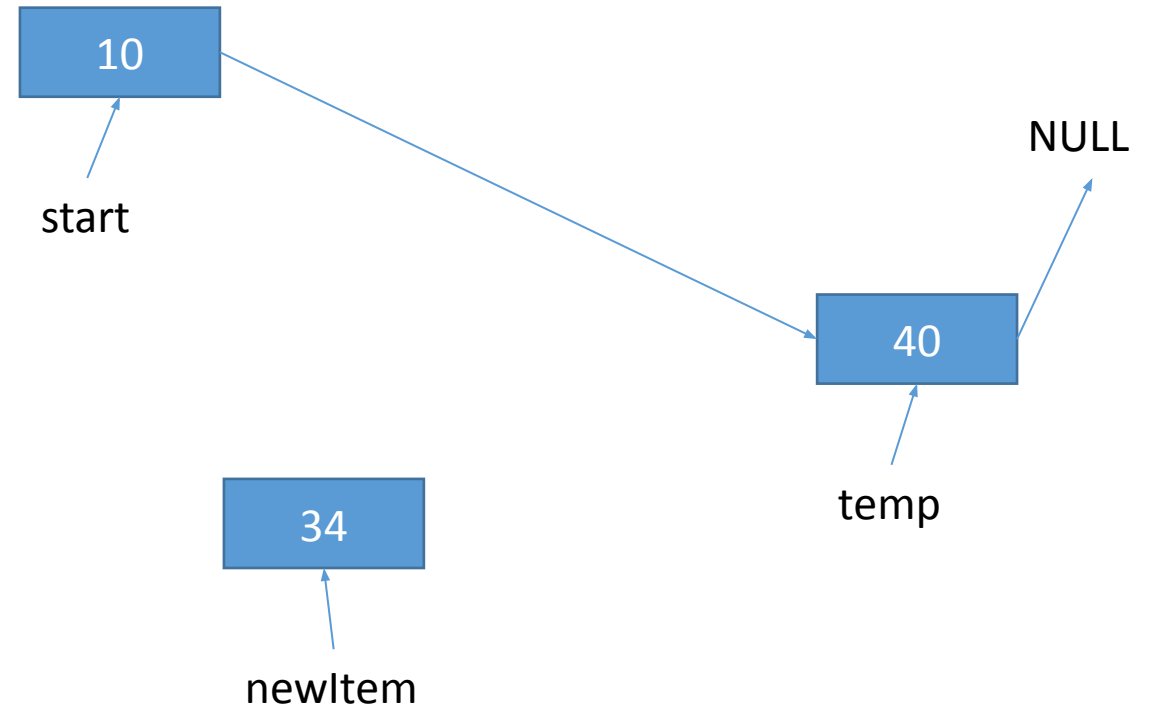
Review Class Linked List

- `start=(node*)malloc(sizeof(node));`
- `temp=(node*)malloc(sizeof(node));`
- `temp1=(node*)malloc(sizeof(node));`
- `start->data =10;`
- `temp->data=40;`
- `temp1->data=30;`
- `start->next=temp1;`
- `start->next->next=temp;`
- `temp->next=NULL;`
- `start->next=temp1->next;`
- `free(temp1);`
- `newitem=(node*)malloc(sizeof(node));`
- `newitem->data=34;`
- `Newitem->next=start->next;`
- `start->next=newitem;`



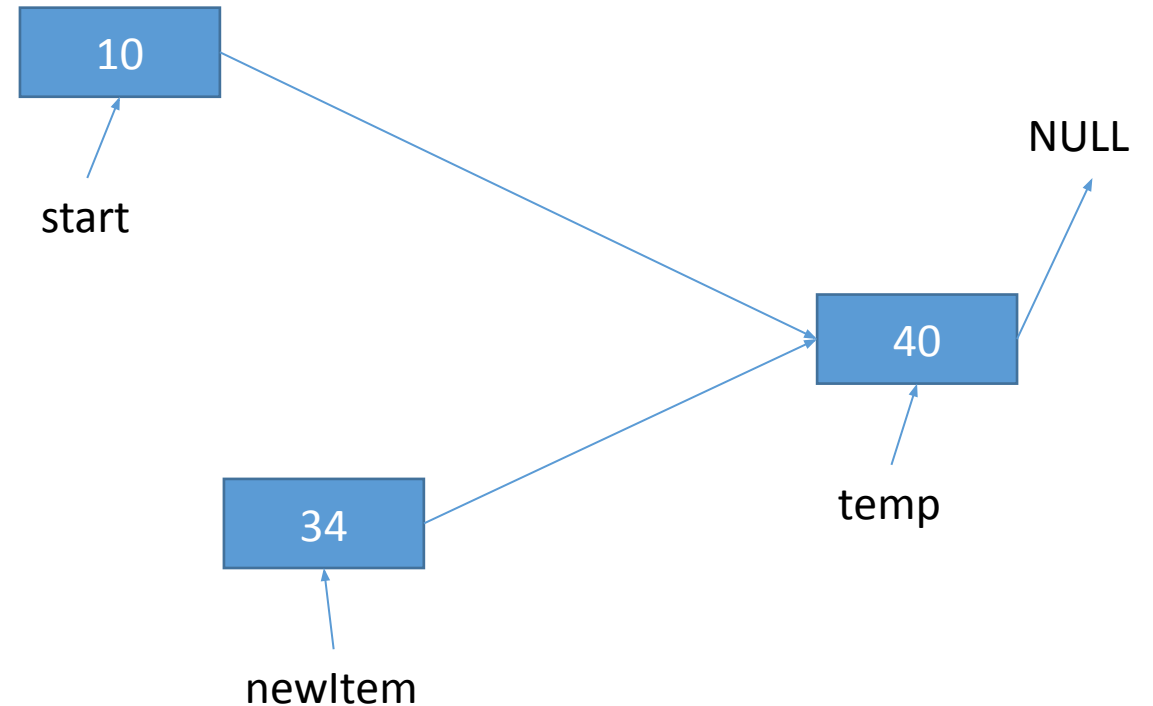
Review Class Linked List

- `start=(node*)malloc(sizeof(node));`
- `temp=(node*)malloc(sizeof(node));`
- `temp1=(node*)malloc(sizeof(node));`
- `start->data =10;`
- `temp->data=40;`
- `temp1->data=30;`
- `start->next=temp1;`
- `start->next->next=temp;`
- `temp->next=NULL;`
- `start->next=temp1->next;`
- `free(temp1);`
- `newitem=(node*)malloc(sizeof(node));`
- `newitem->data=34;`
- `newitem->next=start->next;`
- `start->next=newitem;`



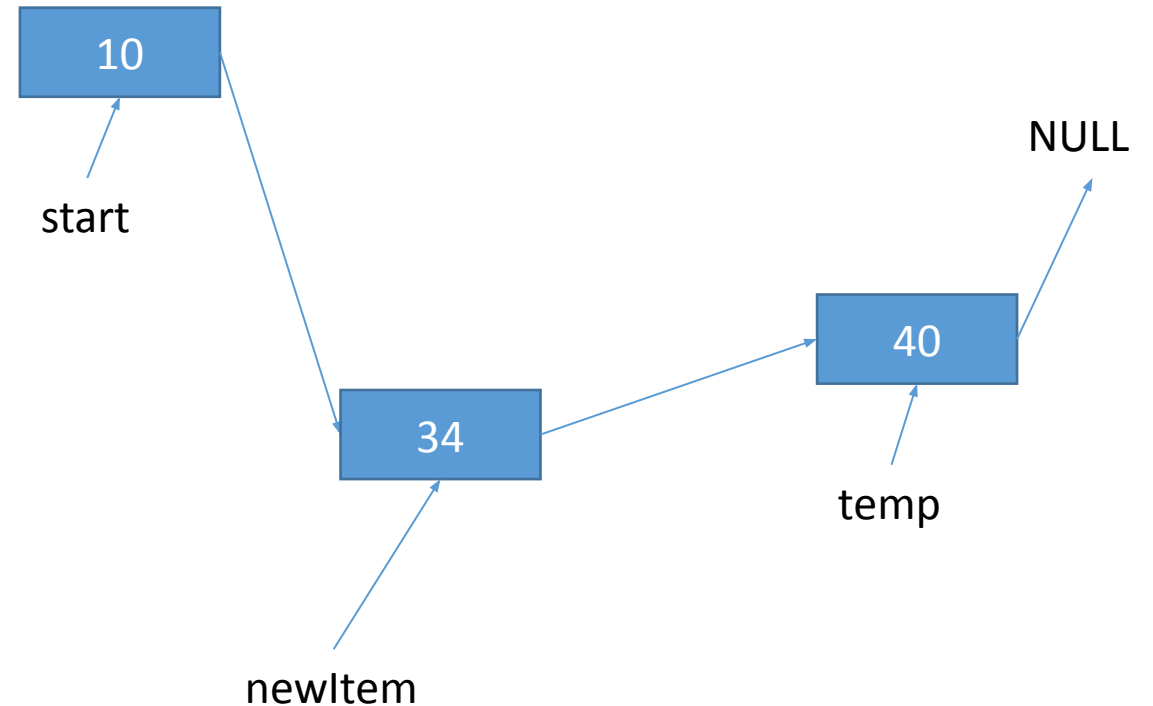
Review Class Linked List

- `start=(node*)malloc(sizeof(node));`
- `temp=(node*)malloc(sizeof(node));`
- `temp1=(node*)malloc(sizeof(node));`
- `start->data =10;`
- `temp->data=40;`
- `temp1->data=30;`
- `start->next=temp1;`
- `start->next->next=temp;`
- `temp->next=NULL;`
- `start->next=temp1->next;`
- `free(temp1);`
- `newitem=(node*)malloc(sizeof(node));`
- `newitem->data=34;`
- `newitem->next=start->next;`
- `start->next=newitem;`



Review Class Linked List

- `start=(node*)malloc(sizeof(node));`
- `temp=(node*)malloc(sizeof(node));`
- `temp1=(node*)malloc(sizeof(node));`
- `start->data =10;`
- `temp->data=40;`
- `temp1->data=30;`
- `start->next=temp1;`
- `start->next->next=temp;`
- `temp->next=NULL;`
- `start->next=temp1->next;`
- `free(temp1);`
- `newitem=(node*)malloc(sizeof(node));`
- `newitem->data=34;`
- `newitem->next=start->next;`
- `start->next=newitem;`



Review Class - Linked List

- Write a function **countElements()** to count the number of elements in a linked list.
- Write a function **searchListByIndex(k)** to search for the kth element of the linked list and return a pointer to it.

Review Class - Linked List

- Write a function **countElements()** to count the number of elements in a linked list.

```
int countElements () {
```

Review Class - Linked List

- Write a function **countElements()** to count the number of elements in a linked list.

```
int countElements() {  
    struct Node traveller=head;  
    if (traveller==NULL) {  
        printf("The list is empty!\n");  
        return -1;  
    }else{  
        int count=0;  
        while(traveller!=NULL) {  
            count++;  
            traveller=traveller->next;  
        }  
        return count;  
    }  
}
```

Review Class - Linked List

- Write a function **searchListByIndex(k)** to search for the kth element of the linked list and return a pointer to it.

```
struct Node* searchListByIndex(int k) {
```

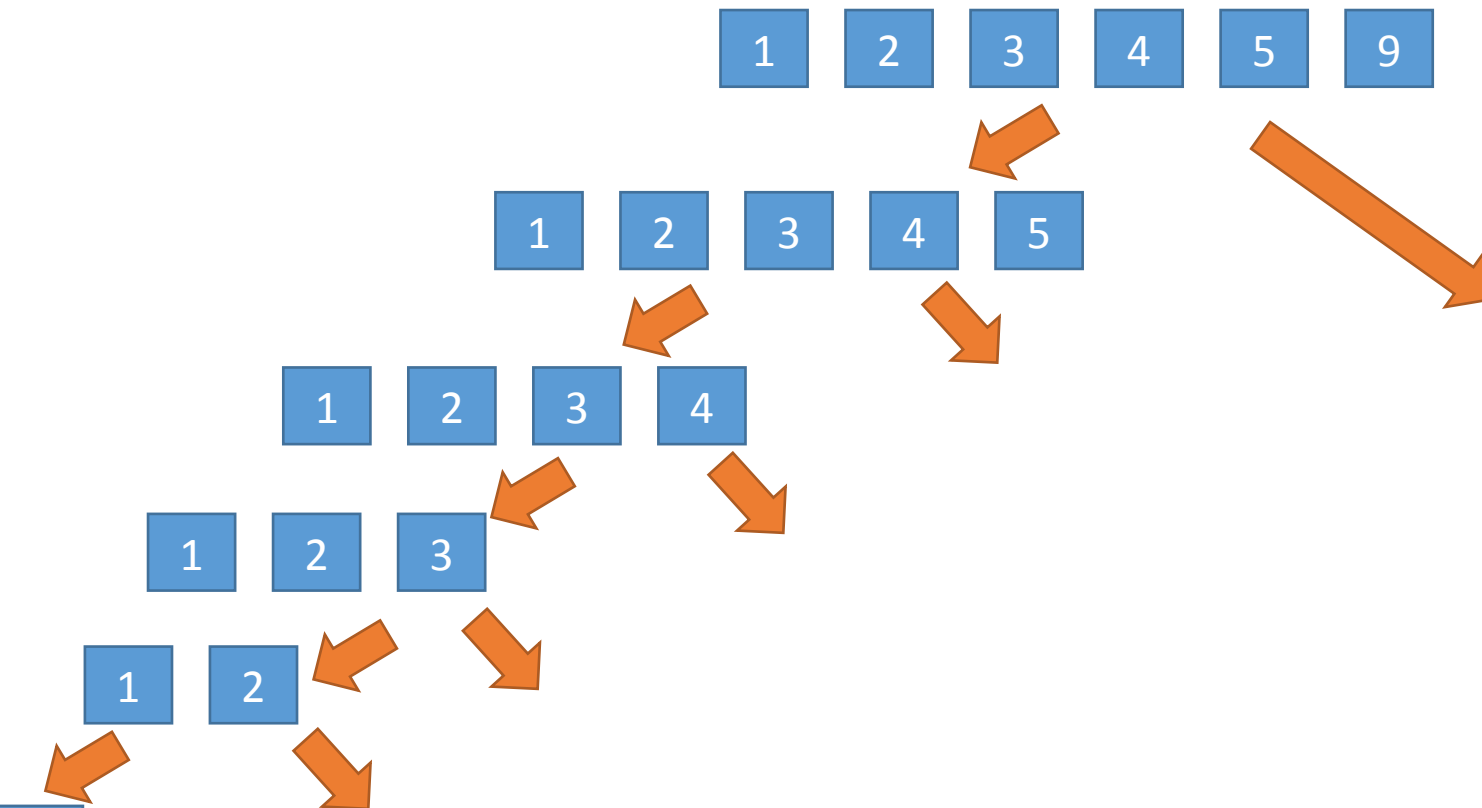
Review Class - Linked List

- Write a function **searchListByIndex(k)** to search for the kth element of the linked list and return a pointer to it.

```
struct Node* searchListByIndex(int k) {
    struct Node* traveller=head;
    if(traveller==NULL) {
        printf("The linked list is empty!\n");
        return NULL;
    }else{
        int index=1;
        while(traveller!=NULL) {
            if(index==k) {
                return traveller;
            }else{
                traveller=traveller->next;
            }
        }
        printf("The list is smaller than the index\n");
        return NULL;
    }
};
```

Review - Quick Sort

- Time Complexity:
- **Worst Case (The partitioning element is the last or the first element)**



QUICKSORT(A, p, r)

```
1  if  $p < r$ 
2       $q = \text{PARTITION}(A, p, r)$ 
3      QUICKSORT( $A, p, q - 1$ )
4      QUICKSORT( $A, q + 1, r$ )
```

	TIME COMPLEXITY
Best Case	$O(N \log N)$
Worst Case	$O(N^2)$

Review - Quick Sort

- Time Complexity:
- **Best Case:** The partitioning element falls at the center.
 - This is exactly like quicksort
 - The array is halved every time

QUICKSORT(A, p, r)

```
1  if  $p < r$ 
2       $q = \text{PARTITION}(A, p, r)$ 
3      QUICKSORT( $A, p, q - 1$ )
4      QUICKSORT( $A, q + 1, r$ )
```

	TIME COMPLEXITY
Best Case	$O(N \log N)$
Worst Case	$O(N^2)$

Review Class

- **Stack and Queue**
- a) Show the **status of a STACK** implemented by an array of size, $m=2$ for the operations given below. Here, x =last digit of your student id+1, $y=x+1$, and $z=y+2$. Push($x+y$), Push($y+z$), Pop(), Push($y*z$), Push($x*y$), Pop(), Pop(), Pop()
- b) Show the **status of a Queue** implemented by a linear (singly) linked list for the following operations. Here, Enqueue and Dequeue mean insert and delete respectively, and x =last digit of your student id+2, $y=x+1$, and $z=y+2$. Enqueue($x+y$), Enqueue($y+z$), Dequeue(), Dequeue(), Dequeue()

Review Class

- For the sorting algorithms:
 - Keep in mind about
 - Ascending Order
 - Descending Order
- Time Complexity:
 - Time complexity from functions (finding c's and n's)
 - Time complexity from algorithms or code snippets



Thank You